

Learn Low-Level Design in 25 Days: Day 2—SOLID Principles Made Simple (with Real Examples)

Ever built a “simple” feature that turned into spaghetti?

TechPreneur

Published October 15, 2025

Free: No

Read on Freedium · original

Contents

What Are SOLID Principles, Really? 0

S — Single Responsibility Principle (SRP) 0

The problem: 0

The fix: 0

O — Open/Closed Principle (OCP) 0

Without OCP: 0

With OCP: 0

L — Liskov Substitution Principle (LSP) 0

Better design: 0

I — Interface Segregation Principle (ISP) 0

Split interfaces: 0

D — Dependency Inversion Principle (DIP) 0

With DIP: 0

Visual Summary 0

Mini Case Study: How Uber Applies SOLID 0

Try It Yourself 0

Key Takeaways 0

What's Next 0

You start with one clean `UserService` class. Then you add logging. Then validation. Then caching. Two weeks later, your "simple" class has 800 lines, 12 dependencies, and breaks every time someone sneezes near the codebase.

That's the moment every developer realizes why **good design principles** matter.

Welcome to **Day 2 of **Learn Low-Level Design in 25 Days** — the hands-on bootcamp where we go from vague diagrams to production-quality class design. Today, we'll explore the **SOLID principles** — five simple guidelines that help you write code that's easy to change without breaking everything else.



By the end of this article, you'll know **how to design flexible, maintainable classes** like senior engineers do — with real-world examples, not textbook definitions.

What Are SOLID Principles, Really?

SOLID isn't magic. It's a mindset — a way to organize your classes so you can:

- Add new features without rewriting old ones
- Fix bugs without introducing five new ones
- Help other developers read your code without asking you to "jump on a quick call"

Let's break them down one by one — not with definitions, but with stories and examples.

S — Single Responsibility Principle (SRP)

> A class should have one reason to change.

The problem:

```
class UserService {
    void registerUser(User user) {
        // Save user
        saveToDB(user);
        // Send welcome email
        sendEmail(user);
    }
}
```

Here, `UserService` does too much: business logic **and** email delivery. When marketing changes the email template, you'll have to touch the same class that handles core user registration. That's risky.

The fix:

```
class UserService {
    private final EmailService emailService;
    void registerUser(User user) {
        saveToDB(user);
        emailService.sendWelcomeEmail(user);
    }
}
```

Now each class has one reason to change. If the email system changes, you only update `EmailService`. That's SRP — simple, but powerful.

O — Open/Closed Principle (OCP)

> Software should be open for extension, but closed for modification.

Without OCP:

```
class PaymentProcessor {
    void process(String type) {
        if (type.equals("CARD")) { /* card logic */ }
        else if (type.equals("UPI")) { /* upi logic */ }
        else if (type.equals("CRYPTO")) { /* crypto logic */ }
    }
}
```

Every time you add a new payment method, you **edit** this class — which increases the risk of breaking old ones.

With OCP:

```
interface Payment {
    void pay();
}

class CardPayment implements Payment { public void pay() { /* card */ } }
class UPIPayment implements Payment { public void pay() { /* upi */ } }

class PaymentProcessor {
    void process(Payment payment) {
        payment.pay();
    }
}
```

Now adding a new payment type doesn't require touching the existing class — just implement a new `Payment`.

That's extension without modification.

L — Liskov Substitution Principle (LSP)

> Subclasses should be substitutable for their base classes.

Imagine this:

```
class Bird {
    void fly() { }
}

class Penguin extends Bird {
    void fly() { throw new UnsupportedOperationException(); }
}
```

If you ever pass a `Penguin` where a `Bird` is expected, your program crashes. That's a violation of LSP.

Better design:

```
interface Bird { }

interface FlyingBird extends Bird {
    void fly();
}

class Sparrow implements FlyingBird { public void fly() { /* ok */ } }
class Penguin implements Bird { }
```

Now `Penguin` doesn't pretend to fly. Your abstraction matches reality.

I — Interface Segregation Principle (ISP)

> Don't force classes to depend on methods they don't use.

Imagine a `Worker` interface:

```
interface Worker {
    void work();
    void eat();
}
```

A robot worker implements `work()` but not `eat()`. That's awkward.

Split interfaces:

```
interface Workable { void work(); }
interface Eatable { void eat(); }

class Human implements Workable, Eatable { ... }
class Robot implements Workable { ... }
```

Each class implements only what it actually needs — clean and focused.

D — Dependency Inversion Principle (DIP)

> Depend on abstractions, not concrete implementations.

Without DIP:

```
class NotificationService {
    private EmailService emailService = new EmailService();
    void notify(User user) {
        emailService.sendEmail(user);
    }
}
```

If tomorrow you switch to SMS, you'll rewrite the class.

With DIP:

```
interface Notifier {
    void notify(User user);
}

class EmailNotifier implements Notifier { ... }
class SMSNotifier implements Notifier { ... }

class NotificationService {
    private final Notifier notifier;
    NotificationService(Notifier notifier) { this.notifier = notifier; }

    void notify(User user) { notifier.notify(user); }
}
```

Now you can inject any notifier you want — **email, SMS, push, even Slack** — without changing `NotificationService`.

Visual Summary



You've just decoupled your system. That's DIP in action.

Mini Case Study: How Uber Applies SOLID

Uber's dispatch system follows SOLID almost religiously:

- Each service (SRP) handles one job: matching, pricing, or location tracking.
- Adding new payment options (OCP) never breaks old ones.
- The driver and rider flows both extend common interfaces (LSP, ISP).
- The notification system (DIP) can switch from email to in-app instantly.

That's how Uber keeps evolving — without constantly rewriting everything.

Try It Yourself

1. **Refactor this:** You have a `ReportGenerator` that prints PDFs and Excel files. Apply OCP — how can you add new formats without editing the class?
2. **Reflect:** Which classes in your current project are doing too much? How can you split responsibilities to make them testable and reusable?

Key Takeaways

- **SRP:** One reason to change
- **OCP:** Extend, don't modify
- **LSP:** Subclasses must behave like parents
- **ISP:** Keep interfaces small and focused
- **DIP:** Depend on abstractions, not concretes

When applied together, these make your design scalable, testable, and future-proof.

What's Next

Tomorrow, in **Day 3**, we'll explore **Interfaces, Abstract Classes, and Inheritance** — and how to use them without falling into over-engineering traps.

If you haven't already, check out the full series here: Learn Low-Level Design in 25 Days — A Practical Bootcamp for Devs (https://freedium-mirror.cfd/https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8)

Save the list to follow the series.